

CS-781 Project Report

Deeptanshu Malu
22B1256
22b1256@iitb.ac.in

Deevyanshu Malu
22B1029
22b1029@iitb.ac.in

1 Objective

The main goal of this project is to employ the VERIFAI toolkit to iteratively re-train a convolutional neural network (CNN) to detect cars on roads (for this project we have used the dataset where either one or no cars are present). After each re-training iteration, the local robustness of the network is evaluated using the $\alpha\beta$ -CROWN framework, which estimates the radius of an L_p -norm ball within which the predicted class label remains unchanged, using which samples are generated through the probabilistic program based sampler, which allows variation of semantic scene features. As re-training progresses, the CNN is expected to improve in both accuracy and robustness. The sampling distribution is modified every iterations, guided by robustness analysis.

2 Plan of Action

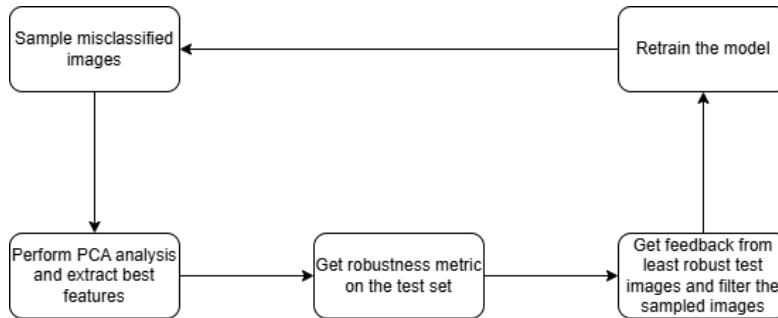


Figure 1: Overall process for increasing robustness for image classification

3 The CNN Model

3.1 Architecture

The CarDetectorModel is a convolutional neural network (CNN) implemented in PyTorch. It consists of three convolutional blocks followed by two fully connected layers. The architecture is shown in Table 1.

Layer	Output Shape	Description
Input	$3 \times 32 \times 32$	RGB input image
Conv1	$8 \times 32 \times 32$	3×3 Conv, padding = 1
MaxPool1	$8 \times 16 \times 16$	2×2 max pooling
ReLU1	$8 \times 16 \times 16$	activation
Conv2	$16 \times 16 \times 16$	3×3 Conv, padding = 1
MaxPool2	$16 \times 8 \times 8$	2×2 max pooling
ReLU2	$16 \times 8 \times 8$	activation
Conv3	$32 \times 8 \times 8$	3×3 Conv, padding = 1
MaxPool3	$32 \times 4 \times 4$	2×2 max pooling
ReLU3	$32 \times 4 \times 4$	activation
Flatten	512	reshape to $(4 \cdot 4 \cdot 32)$
FC1	32	fully connected + ReLU
FC2	2	output logits

Table 1: Architecture of the CarDetectorModel.

3.2 Parameter Analysis

This section provides a detailed breakdown of the number of trainable parameters in each layer of the CarDetectorModel. The network consists of three convolutional layers followed by two fully connected layers.

Layer	Parameters	Calculation
Conv1	224	$(3 \cdot 3 \cdot 3 \cdot 8) + 8$
Conv2	1168	$(3 \cdot 3 \cdot 8 \cdot 16) + 16$
Conv3	4640	$(3 \cdot 3 \cdot 16 \cdot 32) + 32$
FC1	16416	$(512 \cdot 32) + 32$
FC2	66	$(32 \cdot 2) + 2$
Total	22514	$224 + 1168 + 4640 + 16416 + 66$

Table 2: Trainable parameter count and calculations for each layer of the CarDetectorModel.

4 Steps in the Pipeline

4.1 Data Preprocessing and Pre-Training

We first sample 600 images for each class out of which 550 are used for training/validation and the rest of the 50 are used for testing. These will be our training/validation and testing dumps. We then pre-train the model as mentioned before with 200 randomly sampled images from the training/validation images for 10 epochs. Throughout the process the validation data is 20% of the total training data. We reshape the original image of size 1242×375 to 32×32 using LANCZOS resampling for training/validation and testing.

4.2 Falsifier and Classifier

We begin with the **pre-trained model** or the model from the previous iteration. Using this model, misclassified images are identified with the help of VERIFAI. We run the falsifier for 600 iterations and sample at most 200 falsified images randomly for each class.

4.3 PCA Analysis

For each class PCA analysis is performed on the sampled images and the first principal component is returned. Only the continuous space features are considered for this analysis. The 2 high-level features with the greatest loadings are saved. According to [Dreossi et al., 2018], the features with largest weightings have the least robustness.

4.4 $\alpha\beta$ -CROWN Analysis with AutoLiRPA

We perform $\alpha\beta$ -CROWN analysis on each test image in a class to get their robustness metric values. We have taken the $\|\cdot\|_\infty$ norm for $\alpha\beta$ -CROWN. The robustness metric is calculated as in Algorithm 1. The FINDMAXROBUSTEPS function takes as input the CNN model, the image vector, the gold label, the predicted label, the upper limit of the perturbation radius and the tolerance value. This function calculates the largest radius of perturbation around an input image for which the classification output remains unchanged. If the predicted label is not the same as the gold label, negative of the maximum perturbation radius is returned. This output of the function is the robustness metric for an image.

The bottom 20% of test images with the least robustness metric values are taken and their feature values for the previously saved high-level features are obtained. Then for each high-level feature saved in 4.2, the maximum and minimum values are obtained. Finally, the sampled images created in 4.2 are filtered out if those features don't lie in between maximum and minimum values.

4.5 Re-Training

For each class, more images are sampled from the original training/validation in the following way: if the number of images sampled from VERIFAI is greater than 100, we clip it off to 100 images and randomly sample 100 more images from the original training/validation dump, otherwise we sample enough images from the original training/validation dump to get a total of 200 images. Now the previous iteration model or the pre-trained model is re-trained for 5 epochs and saved.

Algorithm 1 Binary Search for Maximum Robust Epsilon

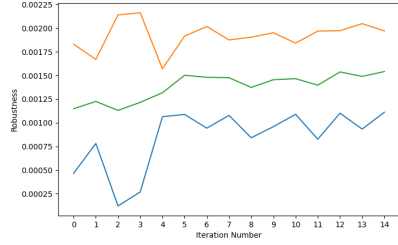
```

1: function CHECKROBUSTNESS( $\mathcal{M}, x, \hat{y}, \epsilon$ )
2:    $(\ell, u) \leftarrow \text{AutoLiRPA}(\mathcal{M}, x, \epsilon)$ 
3:    $\triangleright$  Extract the lower and upper bounds both neurons
4:   Extract  $\ell[0], \ell[1], u[0], u[1]$ 
5:   if  $\hat{y} = 0$  then
6:     return  $(\ell[0] \geq u[1])$ 
7:   else
8:     return  $(\ell[1] \geq u[0])$ 
9:   end if
10: end function

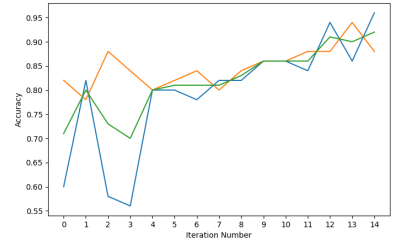
11: function FINDMAXROBUSTEPS( $\mathcal{M}, x, y_{gold}, \hat{y}, \epsilon_{\max} = 0.05, \tau = 10^{-4}$ )
12:    $low \leftarrow 0$ 
13:    $high \leftarrow \epsilon_{\max}$ 
14:    $\epsilon^* \leftarrow 0$ 
15:   while  $high - low > \tau$  do
16:      $mid \leftarrow (low + high)/2$ 
17:     if CHECKROBUSTNESS( $\mathcal{M}, x, \hat{y}, mid$ ) then
18:        $\epsilon^* \leftarrow mid$ 
19:        $low \leftarrow mid$   $\triangleright$  Still robust, try to find a higher  $\epsilon$ 
20:     else
21:        $high \leftarrow mid$   $\triangleright$  Not robust, try to find a lower  $\epsilon$ 
22:     end if
23:   end while
24:   if  $y_{gold} \neq \hat{y}$  then
25:     return  $-\epsilon^*$ 
26:   else
27:     return  $\epsilon^*$ 
28:   end if
29: end function

```

5 Results



Robustness v/s Iteration No.



Accuracy v/s Iteration No.

The blue line represents the robustness metric/accuracy on the images with 0 cars and similarly the orange line is for the images with 1 car. The green line is the average of both the blue and orange line. As seen from the figure, robustness and accuracy both increase with iterations.

References

[Dreossi et al., 2018] Dreossi, T., Ghosh, S., Yue, X., Keutzer, K., Sangiovanni-Vincentelli, A., and Seshia, S. A. (2018). Counterexample-guided data augmentation.